

Building an AI Automation Platform

A complete guide for a team that has *used* n8n and is now going to build one. Every term explained from scratch, every part linked back to something you have already seen, and six phases from a forty-line script to a working demo.

6	15	13	5	Week 14	Week 15
PHASES	STAGES	CORE NODES	TEAM MEMBERS	DEMO	REPORT

PART 0 · PAGES 3–4

The vocabulary

Sixteen words this document keeps using — what each one means, and where you already met it inside n8n.

PART 1 · PAGES 5–11

How it all works

What a node, an item, an engine, a canvas and an API actually are, before we build any of them.

PARTS 2–4 · PAGES 12–26

Building it

Six phases, then integration, testing and the demo. Each with tools, steps and a clear finish line.

Contents

PART 0 – THE VOCABULARY

0.1	The words we will keep using — nodes, items, triggers, canvas, engine	3
0.2	The words behind the screen — server, API, request, JSON, database, credential · Figure 1	4

PART 1 – HOW IT ALL WORKS

1.1	What a workflow really is · Figure 2	5
1.2	What a node really is · Figure 3	6
1.3	What an item really is · Figure 4	7
1.4	What the engine really is · Figure 5	8
1.5	The canvas and the API — the two halves of the app · Figure 6	9
1.6	The six phases · Figure 7	10
1.7	The toolkit — what we install, and when · Figure 8	11

PART 2 – BUILDING IT, PHASE BY PHASE

2.1	Phase 1 · Make it run — the engine and the node shape · Figure 9	12
2.2	Phase 2 · Make it real — database, server and canvas · Figure 10	13
2.3	Phase 3 · Make it usable — self-building forms and expressions · Figures 11–12	14
2.4	Phase 4 · Make it start by itself — webhooks and schedules · Figure 13	15
2.5	Phase 5a · Permission to use a Google account · Figure 14	16
2.6	Phase 5b · The nodes themselves · Figures 15–16	17
2.7	Phase 6 · Ship it — online, measured, written up · Figure 17	18

PART 3 – PUTTING IT TOGETHER

3.1	How six phases become one system · Figure 18	19
3.2	Four levels of testing · Figure 19	20
3.3	Proving it works on the day · Figure 20	21

PART 4 – RUNNING THE PROJECT

4.1	Who owns what · Figure 21	22
4.2	The three risks that end demos, and six habits	23
4.3	What each of us does this week	24

HOW TO READ THIS

Parts 0 and 1 explain how the thing works and are worth reading before anyone writes code — they are deliberately quiet and plain. From Part 2 onwards each phase has its own colour, which is carried through every diagram belonging to that phase, so you can always tell where you are.

0.1 The words we will keep using

Most of the confusion in a project like this comes from words, not from ideas. These eight describe things you have **already seen with your own eyes** in n8n — we are only giving them their proper names so the rest of this document can be short.

WORD	WHAT IT MEANS, PLAINLY	WHERE YOU MET IT IN N8N
Workflow	One complete automation, from the thing that starts it to the last thing it does. It is just a drawing: some boxes and some lines.	Your weather automation. One tab, one name, one on/off switch.
Node	One step in that drawing. A node takes some data in, does one job, and passes data out. Every node works the same way, whether it sends an email or adds two numbers.	Every box you dragged from the panel on the left.
Trigger	A special node with nothing before it. It is what <i>starts</i> a workflow — a clock reaching a certain time, or a message arriving from outside.	Your Schedule Trigger. Also the Webhook node.
Item	One single piece of data travelling between nodes — one row, one email, one order. Nodes always pass a <i>list</i> of items, even when the list has only one thing in it.	The numbered rows in the OUTPUT panel when you clicked a node.
Canvas	The big area where you drag boxes and draw lines between them. It is only a drawing tool — the canvas never runs anything itself.	The grey grid that filled most of the n8n screen.
Execution (a “run”)	One occasion on which a workflow actually ran. Each run is recorded so you can look back at it: what came in, what each node produced, whether it worked.	The Executions list, with green ticks and red crosses.
Expression	A small piece of text inside a settings box, written between double braces, that gets replaced with real data just before the node runs.	<code>{{ \$json.city }}</code> — what appeared when you dragged a field into a box.
Credential	A saved login for an outside service, stored once and reused by any node that needs it. Kept scrambled so that reading the database does not reveal it.	The Credentials screen where you connected your accounts.

WHY THIS MATTERS MORE THAN IT LOOKS

Three of these words carry the whole project. **Node** tells us what four of us will spend Weeks 8–10 writing. **Item** decides the shape of every piece of code we write, and cannot be changed later. And **execution** is the thing our database exists to remember. If those three are clear to everyone, the rest of this document is mostly detail.

0.2 The words behind the screen

These eight describe machinery you **used without seeing**. When you opened n8n at `localhost:5678`, every one of these was already working for you.

WORD	WHAT IT MEANS, PLAINLY	WHERE IT WAS IN N8N
Server	A program that sits waiting and answers when something asks it a question. It does not have a screen; it just listens on a numbered door called a <i>port</i> .	The thing you started, listening on port 5678.
API	The list of questions a server agrees to answer, and the exact wording each one needs. Think of it as a menu: not the kitchen, just what you are allowed to order.	Two of them at once — n8n's own, used by its screen, and the weather service's, used by your HTTP node.
Endpoint	One single item on that menu. An address plus a verb — for example “GET me the list of workflows”, or “POST this new workflow and save it”.	<code>/rest/workflows</code> , called every time you opened the workflow list.
Request & response	The question and the answer. One request goes out, one response comes back, and the conversation is over. Nothing stays connected in between.	Every click you made sent one and waited for the other.
JSON	A plain-text way of writing down data that both people and programs can read. Curly braces for a thing, square brackets for a list of things.	The OUTPUT panel, and the file you get from “Download workflow”.
Database	A program whose only job is to remember things after everything else has been switched off, and to find them again quickly.	Quietly holding your workflows so they were still there tomorrow.
Token	A temporary key that a service hands you after you log in, so that later requests do not need your password. Most expire after about an hour.	What n8n silently refreshed behind your Google credential.
Container (Docker)	A program packed up with everything it needs to run, so it behaves identically on every machine. Started with one command, thrown away with another.	<code>docker run n8nio/n8n</code> — why it worked first time.

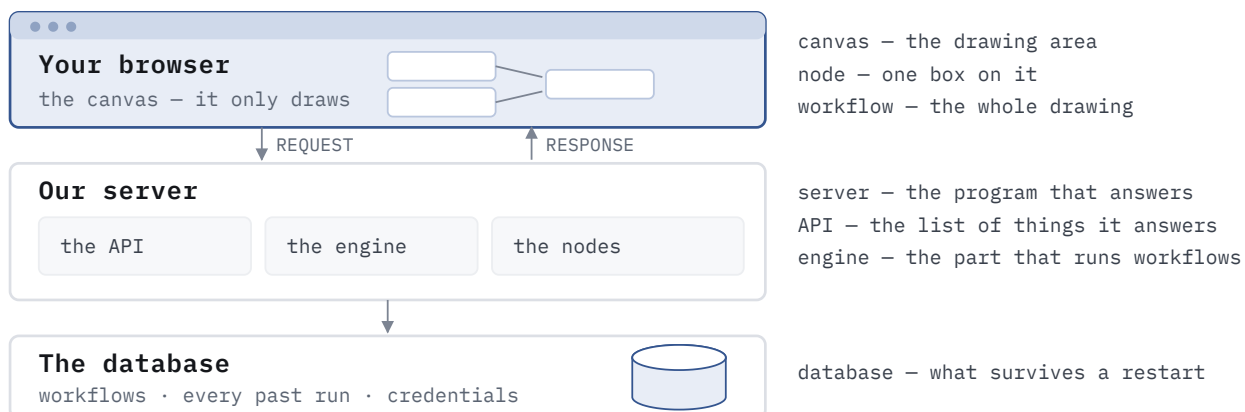


Figure 1. Where every word lives. The browser draws, the server does the work, the database remembers. Almost everything in this document sits in one of these three boxes.

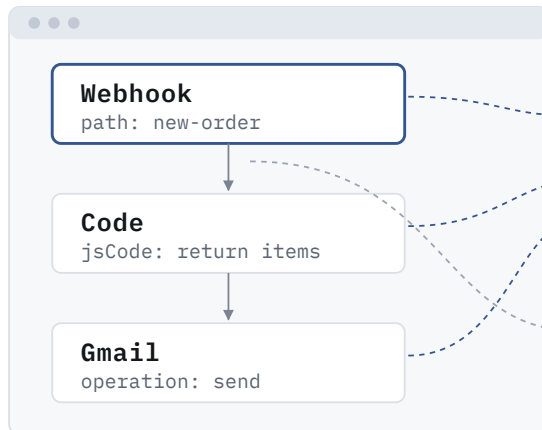
1.1 What a workflow really is

A workflow feels like a picture. It is actually **a piece of text** — a list of boxes and a list of lines, written down in JSON. The picture is only how we look at it.

FAMILIAR FROM N8N

When you clicked “Download” on a workflow, n8n handed you a `.json` file. That file *was* the workflow — you could email it to a friend, they could import it, and the same picture appeared on their screen. Nothing else was needed. That is the single most important thing to understand before we start.

WHAT WE SEE — THE CANVAS



WHAT IS ACTUALLY STORED — ONE PIECE OF TEXT

```
{
  "nodes": [
    { "name": "Webhook", "type": "webhook",
      "parameters": { "path": "new-order" } },
    { "name": "Code", "type": "code", ... },
    { "name": "Gmail", "type": "gmail", ... }
  ],
  "connections": {
    "Webhook": [ [ "Code" ] ],
    "Code": [ [ "Gmail" ] ]
  }
}
```

THE BOXES

THE LINES

Figure 2. Every box you drag becomes one entry in **nodes**; every line you draw becomes one entry in **connections**. That is the whole file format — and it is all the engine ever receives.

WHY THIS IS GOOD NEWS

Because a workflow is just text, the two halves of our project can be built by different people at the same time. The canvas only has to *produce* this text; the engine only has to *read* it. Neither needs to know how the other works — they only have to agree on the shape above.

WHAT IT MEANS FOR US

Our database will store this text in a single column. Saving a workflow is one write; loading it is one read; making a copy is copying a row. Big commercial platforms do exactly the same thing, for exactly this reason.

1.2 What a node really is

A node is **one file that does two separate jobs**. Half of it *describes* itself, so the screen knows what settings to draw. The other half is an ordinary function that *does the work* when the engine calls it. Nothing else. Once this is clear, adding a node to our platform stops being mysterious.

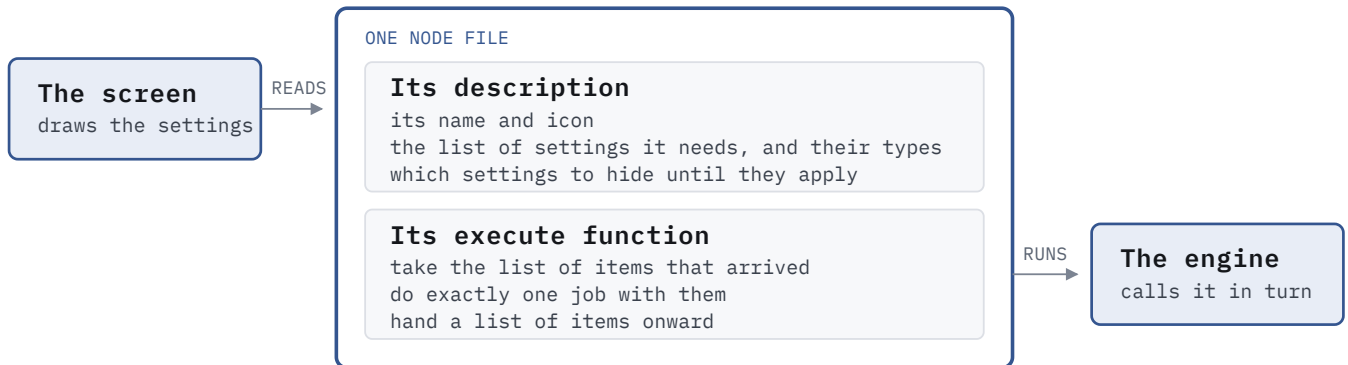


Figure 3. Two halves, two audiences. The screen never runs a node; the engine never draws anything. Neither half needs to know the other exists.

FAMILIAR FROM N8N

Click a Google Sheets node in n8n and the panel on the right appears instantly, with exactly the right fields. Change **Operation** from “Append” to “Read” and the fields change with it. n8n did not hand-build hundreds of panels — every node describes its own fields, and one piece of code turns any such description into a form. We are copying that idea deliberately, and it is the reason four of us can add nodes at the same time.

THE DESCRIPTION HALF

Data, not code

A plain list: “URL is a text box”, “Method is a dropdown with four choices”, “only show Body when Method is POST”. The screen reads that list and draws the panel. Adding a field to a node means adding a line to this list — **never touching the editor**.

THE EXECUTE HALF

One job, done plainly

Items come in, the node does its single job — call an API, reshape some fields, split the flow in two — and items go out. A node never talks to the database, never draws anything, and never decides what runs next. That is the engine's business.

THE RULE THAT KEEPS US OUT OF TROUBLE

Every node in our platform obeys the same shape, whether it sends email through Gmail or simply adds a field. Because the shape never varies, the engine can run a node it has never seen before, and the screen can draw a settings panel for a node written five minutes ago. We agree this shape in Phase 1 and then **freeze it** — changing it later would break everyone's work at once.

1.3 What an item really is

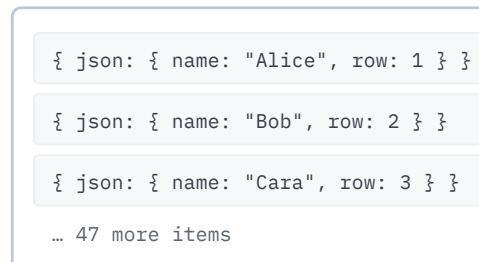
An item is **one thing** — one spreadsheet row, one email, one order. Nodes never pass a single item; they always pass a **list** of them, even when the list is one item long. This sounds like a small detail. It is the decision that shapes every line of code we write.

A REAL SPREADSHEET

name	row
Alice	1
Bob	2
Cara	3
... 47 more	

the Sheets node reads it

BECOMES ONE LIST OF 50 ITEMS



one row becomes one item

AND 50 EMAILS GO OUT



... 47 more
one item, one email

The Gmail node is called **ONCE**. It receives all fifty items and loops over them itself — we never write “repeat this node fifty times” anywhere, because the list already says so.

Figure 4. One read, fifty items, one send node, fifty emails — and no special code anywhere to make that happen.

FAMILIAR FROM N8N

Open any node's **OUTPUT** panel in n8n and the heading says something like “*5 items*”, with little numbered tabs you can click through. That is exactly this list. And it explains something you may have noticed: put a Gmail node after a node that produced five items, and n8n sends five emails without you configuring anything. The list is why.

INSIDE ONE ITEM

A labelled bag of values

Each item holds a `json` part — the ordinary fields, like a name and an email address — and optionally a `binary` part for files riding along, such as a PDF that a later node will attach to an email.

THE LIST OF LISTS

One list per exit

Most nodes have one exit, so they return one list. A node that splits the flow, like IF, has two exits and returns two lists — everything that passed the test, and everything that did not.

THE BUG EVERY ONE OF US WILL HIT AT LEAST ONCE

Returning one list where two were expected, or the other way round. Remember it as: **the outer list is the exits, the inner list is the items**. A normal node returns “one exit, containing all my items”. When a node mysteriously sends nothing onward, or sends everything down the wrong branch, this is almost always why — and printing the data between nodes finds it in about a minute.

1.4 What the engine really is

An **engine** is a program that reads a set of instructions and carries them out in the right order. A chess engine reads a position; a search engine reads a query. **Our engine reads a workflow** — the JSON from section 1.1 — and calls each node's execute function at the right moment, with the right data. It is about forty lines long, and it is the heart of the whole project.

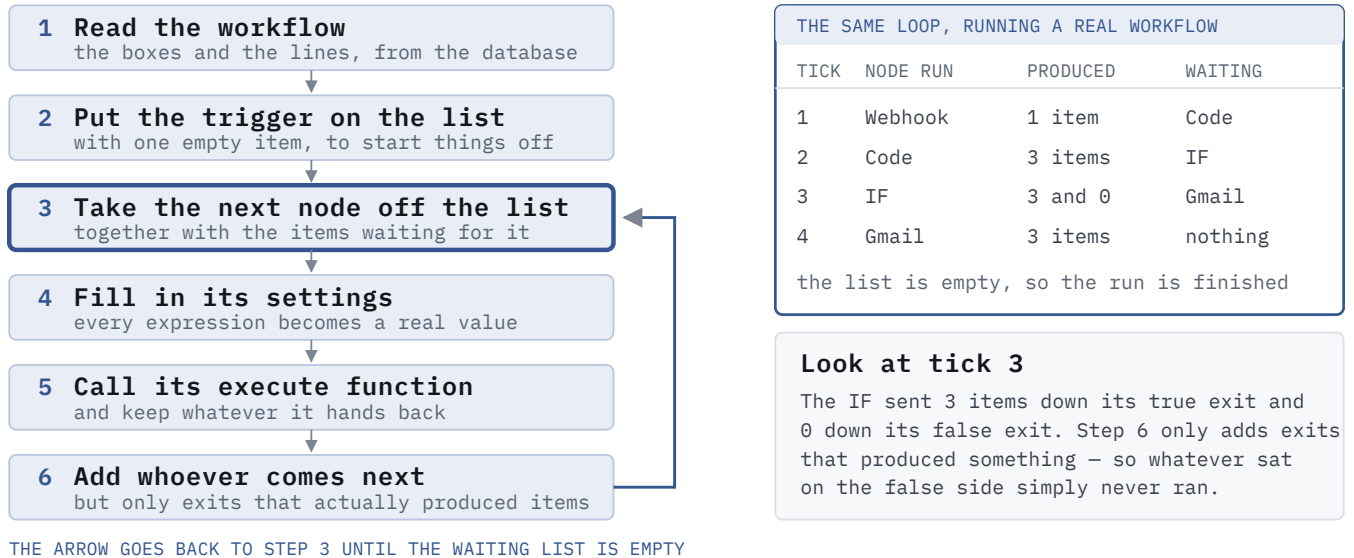


Figure 5. Left: the loop. Right: the same loop actually running our demo workflow, one tick at a time, until the waiting list empties.

FAMILIAR FROM N8N

Press *Execute Workflow* in n8n and you can watch nodes light up one after another, each turning green as it finishes, branches that were not taken staying grey. You were watching this exact loop run. Our version will do the same thing — and because we wrote it, we will be able to explain every step of it in the report.

1.5 The canvas and the API — the two halves of the app

What looks like one application is really **two programs having a conversation**. The **canvas** runs inside your browser and only draws. The **server** runs somewhere else and does all the real work. They never share memory — they only send each other messages, and the list of messages they agree on is called the **API**.

FAMILIAR FROM N8N

Open `localhost:5678` in a second browser window and your workflows are all still there. Close the browser entirely, reopen it — still there. That is because the workflows were never *in* the page. The page asked the server for them, and the server read them from its database. Every screen you saw in n8n was assembled that way.

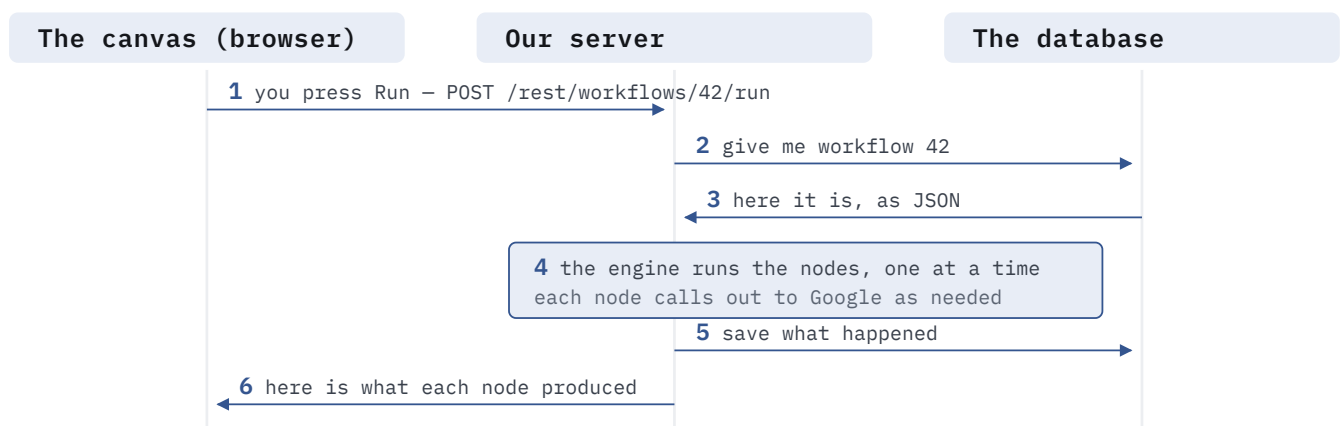


Figure 6. One click, six messages. The canvas never touches the database and never runs a node — it asks, and it displays the answer.

OUR OWN API

Five addresses is enough

List the workflows · fetch one · create one · save one · run one. That is the entire conversation our canvas needs for the first month. Each address has a verb (GET to fetch, POST to create, PATCH to change) and a path such as

`/rest/workflows/42`.

OTHER PEOPLE'S APIS

The same idea, pointed outward

Google publishes an address that means “add this row to that spreadsheet”. Our Sheets node sends a request to it and reads the answer. A node is, almost always, one carefully built request to somebody else's API.

WHY WE CARE ABOUT THIS SPLIT

Because it is what lets two people work at once without colliding. Whoever builds the canvas and whoever builds the server only have to agree on those five addresses — the shape of each message, and what comes back. After that, one can redesign the whole screen and the other can rewrite the whole engine, and neither breaks the other. We write that agreement down in Phase 2 and then leave it alone.

1.6 The six phases

Everything from here on is building. The work splits into six phases, and **each one ends in something we can run and show somebody**. From this page onward each phase carries its own colour, used in its heading and in every diagram that belongs to it, so you can always tell at a glance which part of the project you are looking at.

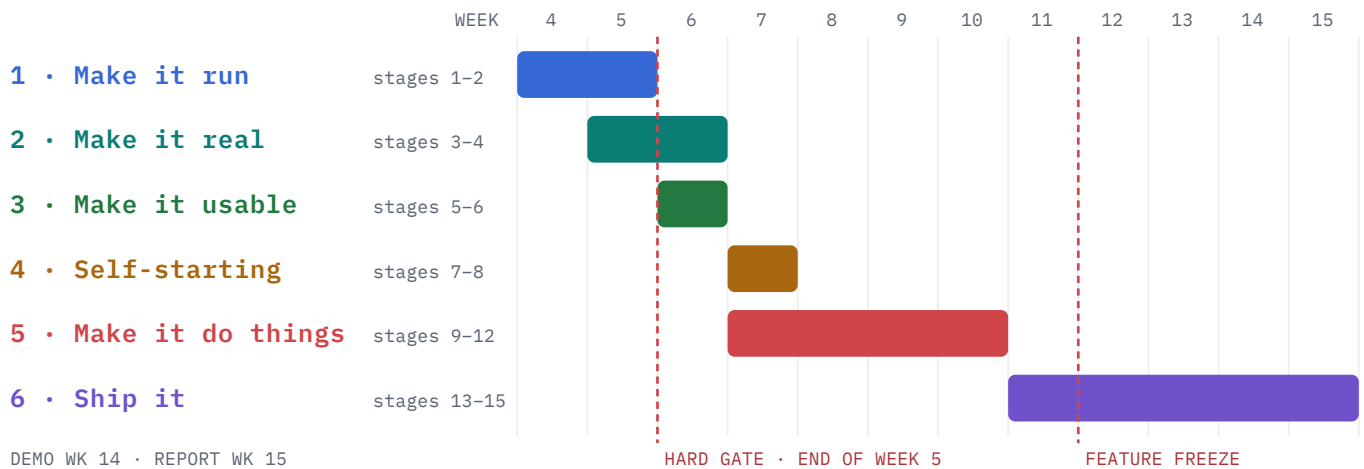


Figure 7. Two dates cannot move. By the end of Week 5 a workflow must run from end to end, however ugly it looks. At the end of Week 11 we stop adding features altogether.

1 · Make it run

A script that executes a workflow and prints the answer. No web page, no database.

2 · Make it real

Workflows are saved in a database and drawn on a canvas.

3 · Make it usable

Settings panels build themselves, and fields can carry live data.

4 · Self-starting

Workflows start on their own, from a web address or from a clock.

5 · Make it do things

Google permission, then the four Google nodes, the Code node and the AI node.

6 · Ship it

Online, measured against n8n, and written up.

THE RULE THAT PROTECTS THE SCHEDULE

If we fall behind we cut *nodes*, never phases. Losing the Drive node costs almost nothing. Losing Phase 3 means every later node needs a hand-built settings panel, and only one person can work at a time — which would cost us the whole of Weeks 8 to 10.

1.7 The toolkit — what we install, and when

The complete list for the semester. **Nobody needs to learn it all now** — each phase introduces two or three things, and about half of them are already built into Node.js. The guiding rule is on the right of the figure: build the thing the assignment is about, and borrow everything else.

We write this ourselves

the engine loop · the node shape · every node
the settings-panel generator · expressions
webhook handling · the schedule registry
the Google permission flow · the sandbox
this is the project — the report comes from it

We deliberately do not

dragging and connecting boxes (React Flow)
the database (Postgres) · HTTPS (Caddy)
running on a schedule (croner)
the code-editing box (Monaco)
solved problems — using them is a good decision

Figure 8. Borrowing the canvas library is not cheating; writing our own would cost a month and earn nothing. Borrowing the *engine* would be the mistake.

TOOL	WHAT IT IS FOR	PHASE
Node.js 22	The language everything is written in — the one we already learned this semester.	1
Git + GitHub	One shared folder with a full history of who changed what. Everyone works on a branch and merges.	1
TypeScript	JavaScript with labels on the data, so a mistake shows up in the editor instead of at three in the morning.	1
Docker Compose	Starts the database and the app together with one command, identically on all five laptops.	2
PostgreSQL 16	The database. Remembers workflows and every past run.	2
Express	The server framework — it turns “when someone asks for this address, do that” into three lines.	2
React + Vite + React Flow	Builds the page in the browser, and gives us the canvas — dragging, drawing lines, panning and zooming, all solved.	2
Monaco	The code-editing box in the sidebar — literally the editor from VS Code.	3
croner	Runs a function on a schedule, with timezones handled properly.	4
Postman / curl + cloudflared	Send test requests to our own addresses, and give our laptop a temporary public address so outside services can reach it.	4
Google Cloud Console	The website where we switch on the four Google APIs and set up the permission screen. Free.	5
node:crypto, worker_threads	Both built into Node. One scrambles the stored Google keys; the other runs a user's JavaScript without risking the server.	5
Anthropic API	The AI node — one request, one response. Ollama can serve a local model instead, as in our weather project.	5
AWS EC2	One small rented computer that runs the finished platform, about \$20 a month.	6
Caddy	Gets and renews the HTTPS certificate by itself. Three lines of configuration.	6
n8n (Docker)	The platform we compare ourselves against, set up the same way as ours so the comparison is fair.	6
k6	Sends measured traffic at both platforms so the comparison uses real numbers.	6

PHASE 1 · MAKE IT RUN

STAGES 1-2 · WEEKS 4-5 · LEAD: ENGINE

2.1 A workflow runs, from a single file

What we are doing. Writing the loop from section 1.4 as one small program, and agreeing the exact shape every node will have. **No web page, no database, no canvas** — just a file we run from the terminal and watch print real results.

```

● ● ● engine.js

$ node engine.js

Start -> [ { } ]

Weather -> [ { "temp_C": "18",
"desc": "Partly cloudy" } ]

Shape -> [ { "city": "Newcastle",
"temp": 18 } ]

done

```

What just happened

A hard-coded workflow object, three node functions, and the loop — all in one file, about forty lines.

It called a real weather API, passed the result to the next node, and printed what each one produced.

everything else is built around this

Figure 9. The whole product, on the first day. Every later phase adds a way to create, store, trigger or display this — never a second way to run it.

FAMILIAR FROM N8N

This is the moment n8n goes from magic to machinery. What you watched happen on the n8n canvas — nodes lighting up in order, data appearing in each output panel — is exactly what those printed lines are. The only difference is that n8n draws it prettily and we are printing it to a terminal.

TOOLS WE USE IN THIS PHASE

Node.js	the language we already learned — nothing new to install	VS Code + Git	one shared repository, everyone on a branch
TypeScript	used lightly, only to write down the node shape	No frameworks	deliberately — this phase is plain JavaScript in one file

HOW WE PROCEED

1. **One person writes the file.** A hard-coded workflow, two or three node functions, and the waiting-list loop. It should end by printing real data from a public weather API.
2. **All five of us read it together** until nobody has a question. This is worth an hour of everyone's time — it is the only genuinely conceptual part of the whole project.
3. **We write the node shape down** in the repository: what a node receives, what it must hand back, and what it is never allowed to do (touch the database, draw anything, or decide what runs next).
4. **We build two boring nodes against it** — one that adds fields to each item, one that splits the flow in two. Nothing that can fail for reasons outside our control.
5. **We freeze the shape.** From here on, changing it needs everyone to agree, because a change breaks work already in progress.

DONE WHEN

Three nodes run in a chain, the splitting node sends data down the correct branch, and every one of us can explain the loop without notes.

WATCH OUT

Do not put five people on this. One writes, four read. Parallel work on an unstable core produces five rewrites — the most common way a project like ours fails quietly in week three.

PHASE 2 · MAKE IT REAL

STAGES 3-4 · WEEKS 5-6 · LEAD: SERVER + CANVAS

2.2 Workflows survive a restart, and we can draw them

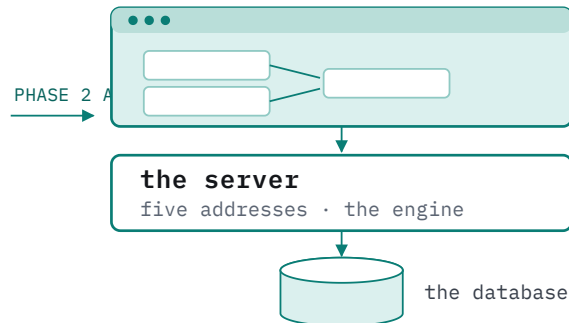
What we are doing. Giving the engine a home. A **database** remembers workflows, a **server** hands them out through the five addresses from section 1.5, and a **canvas** lets us draw one instead of typing JSON by hand.

AFTER PHASE 1

```
$ node engine.js
Start -> [ { } ]
Weather -> [ ... ]
done
```

one file, hard-coded,
gone when it finishes

AFTER PHASE 2



What is new

a page you can draw on
five addresses to call
two tables that remember
a Run button that works

the engine itself does not
change at all – it is simply
called from somewhere new

Figure 10. Phase 2 wraps phase 1 in the four things every web application has: a screen, a server, a database, and a way back. Nothing about the engine is rewritten.

FAMILIAR FROM N8N

Everything on this page is what n8n already gave you for free: a canvas that saved when you clicked away, a workflow list that survived a restart, and a Run button. You never thought about the database behind it – which is exactly the experience we are now building for ourselves.

TOOLS WE USE IN THIS PHASE

Docker Compose	starts the database with one command — nobody installs Postgres by hand	PostgreSQL	remembers the workflows and every past run
Express	the smallest useful server — readable end to end in an afternoon	node-postgres	talks to the database in plain SQL; no extra layer to learn
React + Vite	builds the page the canvas lives in	React Flow	dragging, connecting, panning and zooming — already solved

HOW WE PROCEED

1. **Start the database in a container.** One configuration file, one command, the same on every laptop.
2. **Two tables only:** one for workflows, one for runs. The whole drawing goes into a single column as the text from Figure 2 — which is what makes saving one write and loading one read.
3. **Build the five addresses:** list, fetch, create, save, run. That really is enough for the next month.
4. **Put React Flow on the page.** We write the boxes and the sidebar; we do not write dragging, zooming or line-drawing.
5. **Join the two halves** so the drawing saves to the database and the Run button executes it, showing what each node produced.

DONE WHEN

We can drag out two nodes, connect them, refresh the browser and find them still there — then press Run and see the result of each one.

WATCH OUT

Converting between React Flow's format and ours is the fiddly job here, and essentially every canvas bug for the rest of the semester will live in it. Write it once, carefully, with someone reading over your shoulder.

2.3 Nodes describe themselves, and fields carry live data

What we are doing. The two mechanisms that look small and decide everything: the settings panel that **draws itself**, and **expressions**.

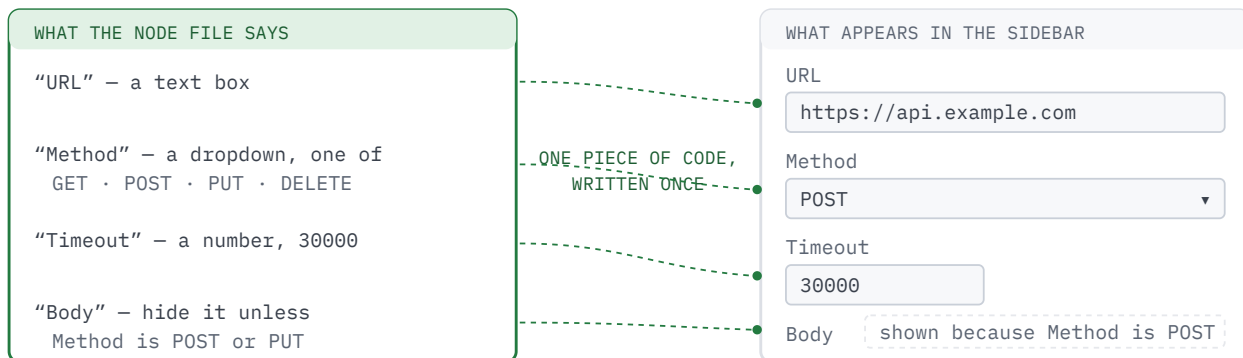


Figure 11. Four lines of description, four fields on screen — and the fourth appears only because Method happens to be POST. Nobody hand-builds a panel, ever.

AND ONE FIELD BECOMES A DIFFERENT VALUE FOR EACH ITEM

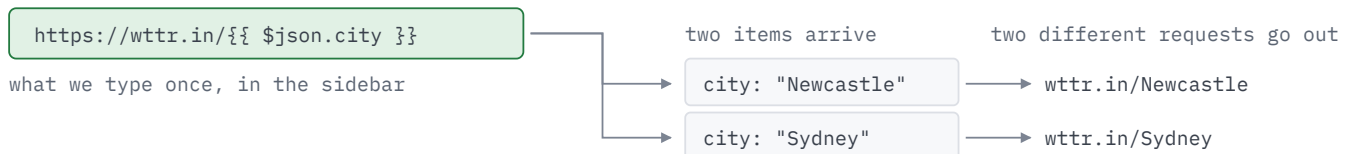


Figure 12. The engine fills the field in for every item separately, just before the node runs — the node never sees the braces.

FAMILIAR FROM N8N

Changing a node's **Operation** made half the fields disappear — that is the top figure. Dragging a field into a box inserted `{{ $json.something }}` — that is the bottom one.

TOOLS WE USE IN THIS PHASE

React	the one component that turns a description into a form	Our own code	no library does this for us — about a hundred lines, and the cleverest part of the project
Monaco	the editor from VS Code, dropped in later as the Code node's text box		

HOW WE PROCEED

1. **Agree the kinds of setting we support** — text, number, on/off, dropdown, code box — then write the single component that reads a description and produces those inputs.
2. **Add “only show this when...”**, so a panel changes as you pick an operation — this is what makes the Google nodes feel finished rather than overwhelming.
3. **Add expressions**, filled in for each item just before the node runs.

DONE WHEN

A brand-new node file appears in the sidebar with a full settings panel, and nobody touched the canvas code.

WATCH OUT

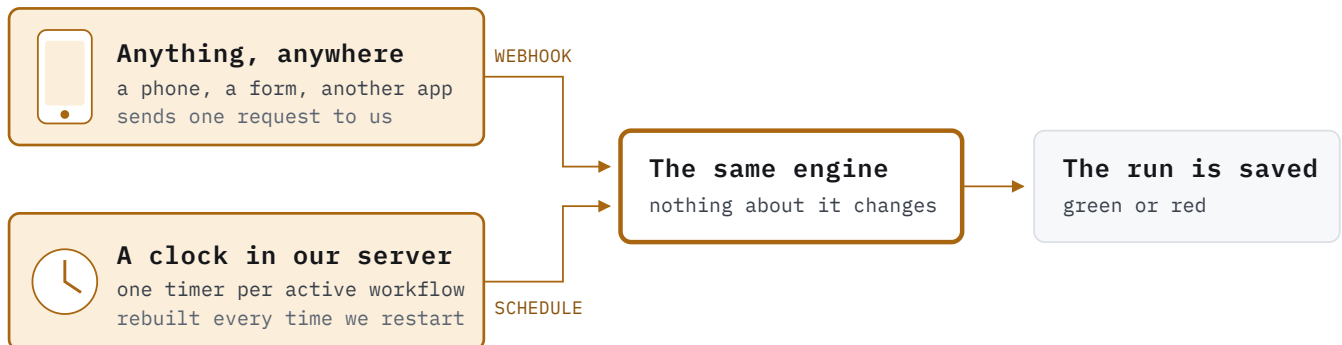
Skip this “to save time” and each of the next ten nodes needs a hand-built panel — turning Weeks 8–10 into a queue behind one person.

PHASE 4 · MAKE IT START BY ITSELF

STAGES 7-8 · WEEK 7 · LEAD: SERVER

2.4 Workflows that run without anyone pressing a button

What we are doing. Building the two **triggers** from section 0.1. One is a **web address** that anything on the internet can call; the other is a **clock** inside our own server. These are the two the assignment names, and together they are what turns a tool into a platform.



Each workflow gets two addresses, exactly like n8n: a test one while we are editing it, and the real one once it has been switched on.

Figure 13. Both routes arrive at the same loop. Adding a trigger never means changing the engine — that is the payoff for getting Phase 1 right.

FAMILIAR FROM N8N

The Webhook node in n8n showed you two URLs, a Test one and a Production one, and made you press “Listen for test event” before the test one would work. That is precisely the mechanism on this page, and we are copying the two-URL idea because it stops us sending real emails while we are still experimenting.

TOOLS WE USE IN THIS PHASE

croner	one small library that runs a function on a schedule, with timezones handled	Postman or curl	to fire test requests at our own addresses without a real caller
cloudflared	a free tunnel giving our laptop a public address while we develop		

HOW WE PROCEED

1. **Give each workflow its own web address.** When a request arrives we look up which workflow owns that address, and start it with the request handed in as the first item.
2. **Two addresses per workflow** — a test one while editing, a live one once activated.
3. **Decide what the caller gets back:** an instant “received”, or wait for the workflow to finish and return its result. Both are useful, and both are easy once the first works.
4. **For schedules, keep one timer per active workflow**, and rebuild the whole set whenever the server restarts — otherwise schedules stop silently after every deploy.
5. **Always store a timezone.** Our laptops are on Australian time and the server will be on UTC, so a workflow set for 9am would otherwise fire in the evening.

DONE WHEN

A request sent from someone's phone runs a workflow on the laptop, and a scheduled workflow produces exactly one run per minute — stopping the moment it is switched off.

WATCH OUT

To test from outside while developing, use a tunnel rather than trying to open ports on the university network. It is one command and saves an afternoon.

PHASE 5 · MAKE IT DO THINGS

STAGE 9 · WEEKS 7-8 · LEAD: SERVER

2.5 First, permission to use a Google account

What we are doing. Getting permission to act for somebody — **without ever seeing their password**. The step most likely to cost us a week, so we do it early and once.

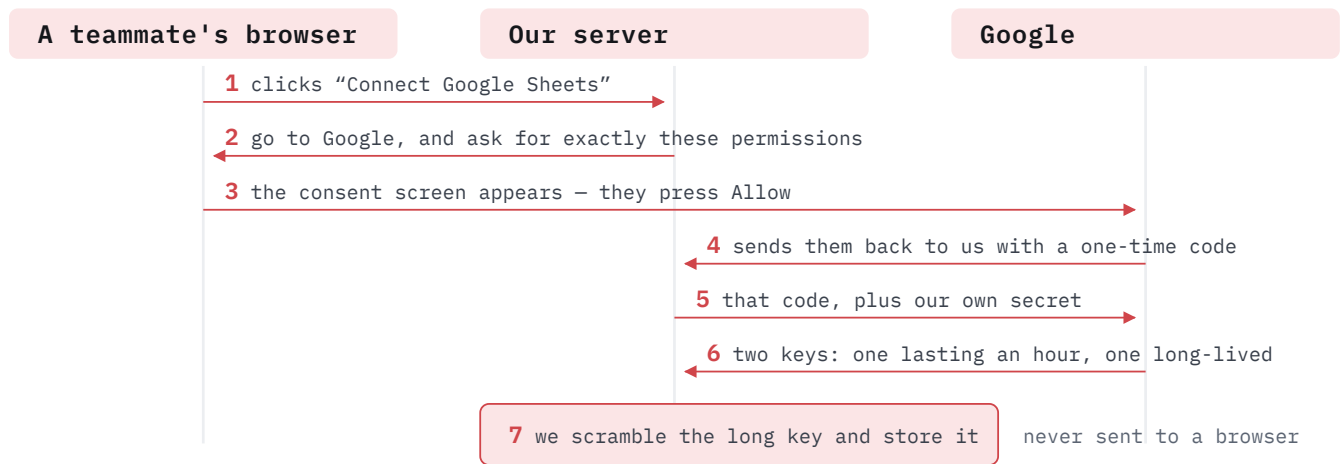


Figure 14. The whole thing in seven arrows. Steps 1–3 happen in front of the person; 4–7 happen invisibly in about half a second.

FAMILIAR FROM N8N

Adding a Google credential opened a small window, showed Google's "choose an account" screen, then said *Account connected*. These seven arrows are what happened inside it.

TOOLS WE USE IN THIS PHASE

Google Cloud Console	where the project, the four APIs and the permission screen live — free	OAuth 2.0	the agreed way of saying "let this app act for me" — a sequence of requests, not a library
node:crypto	built into Node; scrambles the stored keys		

HOW WE PROCEED

1. **One of us creates a Google Cloud project** and switches on the four APIs. Free, fifteen minutes.
2. **Fill in the consent screen** and keep the app in **Testing** mode. Add every group member as a **test user**, plus our demonstrator: in Testing mode nobody outside that list can connect at all.
3. **Build the round trip** from the figure above, then scramble the long-lived key before storing it — and never send it to a browser.

THE TWO TRAPS THAT END DEMOS

1 · Gmail permissions are "restricted".

Publishing the app would need a Google security review we cannot finish this semester — and we do not need one: Testing mode plus a test-user list behaves identically for us.

2 · In Testing mode the long-lived key dies after seven days.

An account connected in Week 9 is dead by Week 14, and it looks like a broken node. **Reconnecting every account is step one of the demo-morning checklist.**

DONE WHEN

Somebody presses Connect, approves on Google's screen, and our server can read a real spreadsheet with the stored keys.

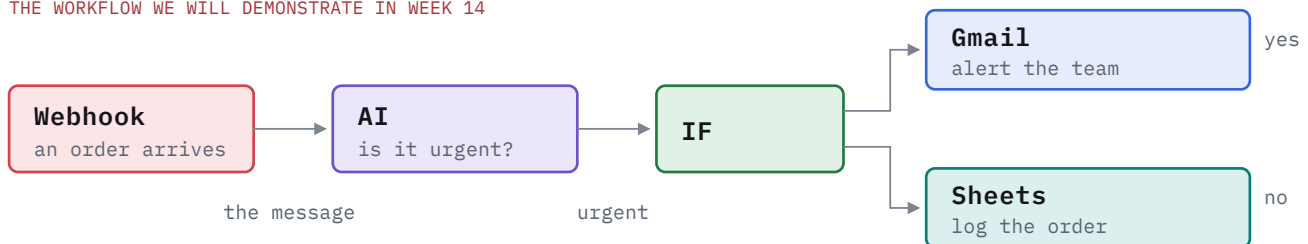
PHASE 5 · MAKE IT DO THINGS

STAGES 10–12 · WEEKS 8–10 · LEAD: INTEGRATIONS

2.6 Then the nodes people actually came for

What we are doing. Building the four Google nodes, the Code node and the AI node. With Phases 1–4 finished, each of these is a small, self-contained job — which is exactly why four of us can now work at the same time without getting in each other's way.

THE WORKFLOW WE WILL DEMONSTRATE IN WEEK 14



Six nodes, five different phases, one screen. Everything in this document exists to make this work.

Figure 15. Each box is coloured by the phase that built it — which is also a picture of how the whole project fits together.

FAMILIAR FROM N8N

This is your weather workflow, grown up. Same shape: something starts it, something thinks about the data, something decides, and something delivers. The only new idea is the branch — and n8n's IF node worked exactly the same way.

HOW WE PROCEED

1. **Build the Google nodes easiest first: Sheets, then Docs, then Drive, then Gmail.** Each is a settings description plus one carefully built request. A single shared helper deals with expired keys, so no individual node has to think about it.
2. **Allow a full day for Gmail.** Sending mail means assembling the message in an old format by hand — knowing that in advance stops it feeling like failure.
3. **Use the Docs node for the best demo moment:** a template document with placeholders, filled in from a spreadsheet row. A mail merge in four nodes, and it looks impressive in ten seconds.
4. **The Code node's work is not running the code** — it is stopping bad code from taking the server down, which is the figure below.
5. **The AI node is one request and about half a day.** Because settings fields carry live data, somebody can type “Classify this message: `{{ ... }}`” straight into the sidebar and feed the answer to an IF node. Do not leave this until last: it is what makes ours an *AI* automation platform.

THE CODE NODE — THREE WALLS, AND AN HONEST ADMISSION

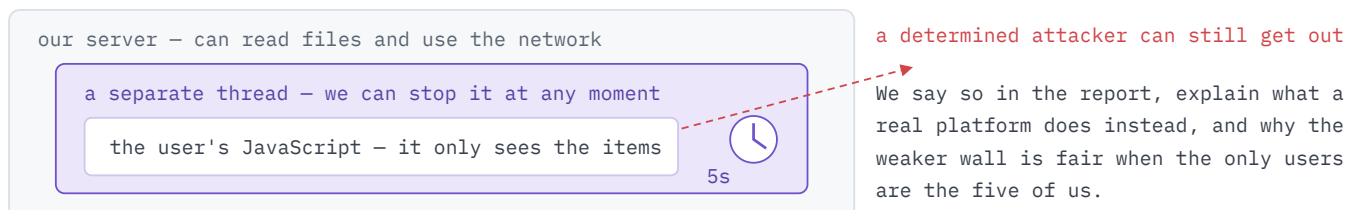


Figure 16. Being able to explain the limits of our own sandbox is worth more marks than quietly using a stronger one would have been.

DONE WHEN

The workflow above runs green from end to end, with a real email arriving and a real spreadsheet row appearing.

PHASE 6 · SHIP IT

STAGES 13–15 · WEEKS 11–15 · LEAD: INFRASTRUCTURE + REPORT

2.7 Online, measured, and written up

What we are doing. Putting the platform on a real computer so our web addresses work from anywhere, then measuring it honestly against n8n, then writing the report. **At the end of Week 11 we stop adding features** — everything after that is fixing, measuring and writing.

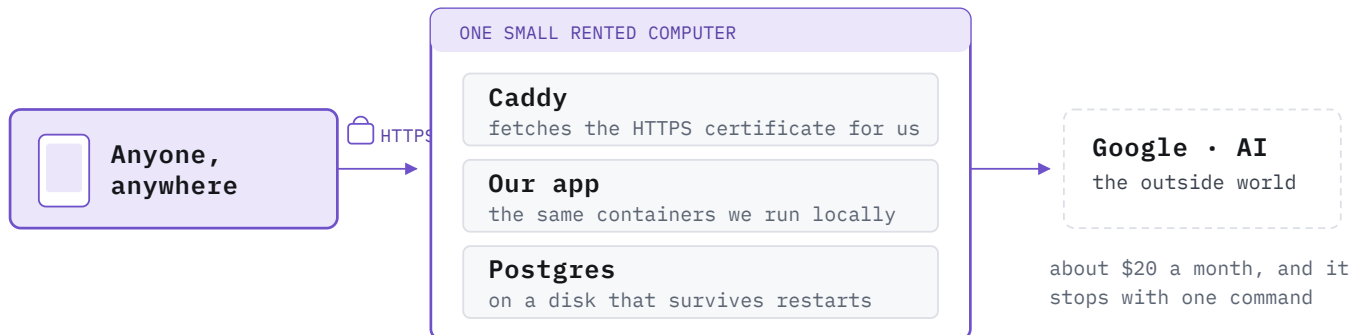


Figure 17. Not a cloud architecture diagram — one machine running the same three containers we already use. An afternoon's work, not two weeks.

FAMILIAR FROM N8N

This is simply n8n's own self-hosting story pointed at our code. The reason your `docker run n8nio/n8n` worked first time is the reason our deploy will: the container carries everything it needs with it.

TOOLS WE USE IN THIS PHASE

AWS EC2	one small rented machine that runs the whole platform	Caddy	gets and renews the HTTPS certificate by itself; three lines of configuration
n8n (Docker)	the platform we compare against, set up the same way as ours	k6	sends measured traffic at both so the comparison is real

HOW WE PROCEED

1. **Freeze features at the end of Week 11.** A group decision made in advance, so it is not an argument later.
2. **Rent one small machine, point a cheap domain at it,** and let Caddy handle HTTPS. Set a spending alarm on day one, before launching anything.
3. **Run n8n on the same machine, configured the same way** — not its simplest setup, or the comparison is unfair and a marker will notice.
4. **Build the same three workflows on both,** run each three times, and keep the raw numbers rather than only the averages.
5. **Time ourselves adding a new node to each platform.** We will win this clearly, and nobody else will think to measure it.
6. **Write honestly.** n8n does far more per run than we do, so being faster mostly means “we do less”. Saying that plainly is worth more marks than the number itself.

DONE WHEN

The canvas and a webhook address both work from a phone on mobile data, the benchmark numbers are recorded, and the report draft is circulating.

WATCH OUT

Write the report from Week 5 onwards, a section at a time, and screenshot everything as it is built. You cannot re-screenshot a server you have already shut down.

3.1 How six phases become one working system

Each phase produced a piece, and each of us has been working in our own folder. Integration is not a week at the end — it is a habit from Week 4: **one repository, one command, and everything runs**. If assembling the project is ever difficult, something went wrong earlier.

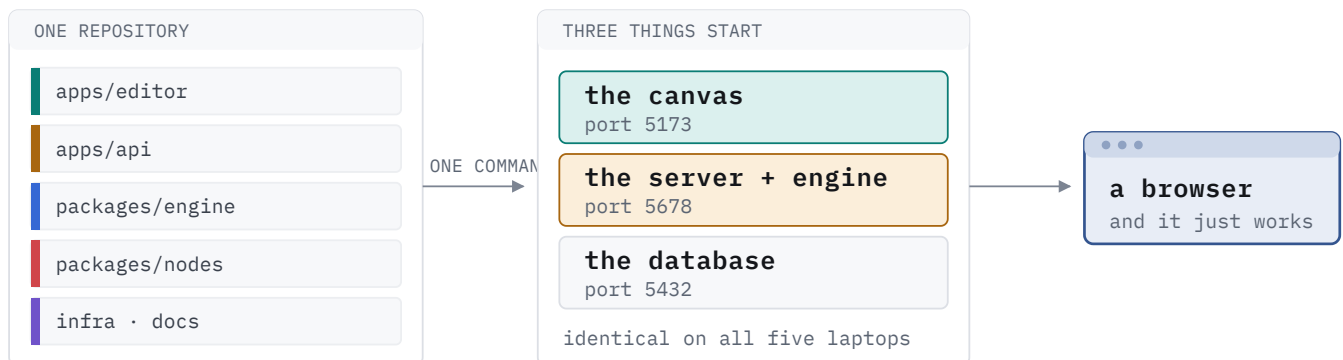


Figure 18. Five folders, one command, three programs. Anyone who can clone the repository can run the whole platform.

HOW WE PROCEED

1. **One repository from day one**, everyone on their own branch, merging into the main one. Never a zip file passed around, and never “I will send you my folder”.
2. **One command starts everything**. If that command ever stops working, fixing it matters more than any feature.
3. **One example settings file** in the repository listing every key the project needs, with fake values. Real keys live only in each person's own untracked copy.
4. **A starter workflow loaded on first run**, so a fresh clone shows something working instead of an empty screen.
5. **Integration every Thursday**. Everything merges, and one person — a different one each week — clones the repository fresh and runs it. Whatever breaks is that week's priority.

DONE WHEN

Someone who has never touched the project can clone it, run one command, and have a working platform in under ten minutes without asking a question.

WATCH OUT

The classic failure is five people whose parts each work alone and have never run together. Merging weekly from Week 4 makes integration boring — which is exactly what we want it to be.

3.2 Four levels of testing, and what each one catches

Not testing everything — testing the things that break quietly. The rule: **the lower the level, the more tests and the faster they run.** We do not need all four from day one; each arrives with the phase that needs it.

1 • Engine tests

about 20 • milliseconds

Does the loop run nodes in the right order? Does a branch with zero items correctly stop everything after it?

2 • Node tests

one per node • seconds

Does the Gmail node build the right request from its settings? Runs against a fake server, never real Google.

3 • Browser test

one or two • a minute

Can a person drag out two nodes, connect them, press Run and see the result? One test covering the whole path.

4 • The real run

by hand • before demos

Does a real email actually arrive? Does a row appear in a real spreadsheet? Only a person can confirm this.

Levels 1–3 run automatically on every change. Level 4 is a checklist we work through before every demo.

Figure 19. Each level catches a kind of bug the level above it cannot. Skipping level 2 is why integrations break silently the week before a demo.

TOOLS WE USE FOR TESTING

Vitest

runs levels 1 and 2 — fast, and the default for a Node project

A fake server

a tiny local program that answers like Google does, so node tests never touch the real thing

Playwright

drives a real browser for level 3

GitHub Actions

runs levels 1–3 on every change, so nobody has to remember

HOW WE PROCEED

1. **Start with the engine (level 1), in Phase 1.** The cheapest tests we will ever write, and they protect the piece everything else depends on.
2. **Add one test per node as we build it (level 2),** not afterwards. The trick is the fake server: our node calls a local address instead of Google's, so the test is fast, free, and works with no internet.
3. **Write exactly one browser test (level 3)** once the canvas can save. One is enough — browser tests are slow and break for boring reasons.
4. **Keep level 4 as a written checklist,** not as tests. Section 3.3 is that checklist.
5. **Turn the automatic checks on early,** so that from Week 5 a broken change is caught when it is pushed rather than the night before the demo.

DONE WHEN

Someone pushes a change that breaks the IF node and a red cross appears within two minutes — without anybody noticing by hand.

DO NOT TEST

Google's APIs, React Flow, or Postgres. They are not our code and testing them wastes time we do not have. Test *our* logic and use a fake for everything else.

3.3 Proving the whole thing works, on the day

The last two weeks are not for building. They are for making sure that what we built works **in front of somebody else, on a day we do not control.**

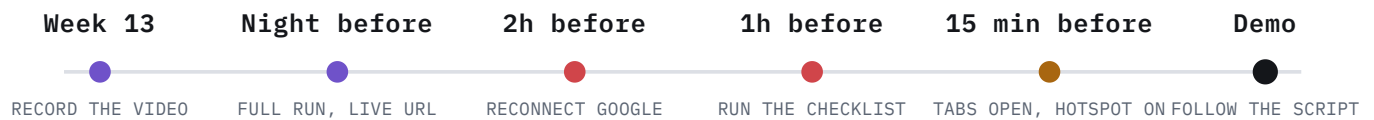


Figure 20. The two red markers are the ones that have ended other groups' demos. Neither takes more than fifteen minutes.

THE ACCEPTANCE CHECKLIST — EVERYTHING THE BRIEF ASKS FOR, AND HOW WE PROVE IT

WHAT IS REQUIRED	WHAT WE SHOW
Schedule trigger	A workflow set to every minute; the run list fills up while we talk, and stops when we switch it off.
Webhook	Somebody in the room sends a request from their phone; the run appears on screen immediately.
HTTP request	The demo workflow calls a real outside API and shows what came back.
Google Sheets	A row appears in a spreadsheet that is open on screen.
Google Docs	A template document comes back with its placeholders filled in from that row.
Google Drive	That finished document appears in a Drive folder.
Gmail	A real email arrives in somebody's inbox, with the document attached.
Code node	A snippet turns one item into ten; then an endless loop is stopped after five seconds and the server stays up.
AI node	The model classifies an incoming message and the IF node routes it to the right branch.
Extensibility	A brand-new node is written live in about ninety seconds and appears with a full settings panel.
Comparison with n8n	One slide: our numbers beside n8n's, the raw data available, and an honest sentence about why we are faster.

DONE WHEN

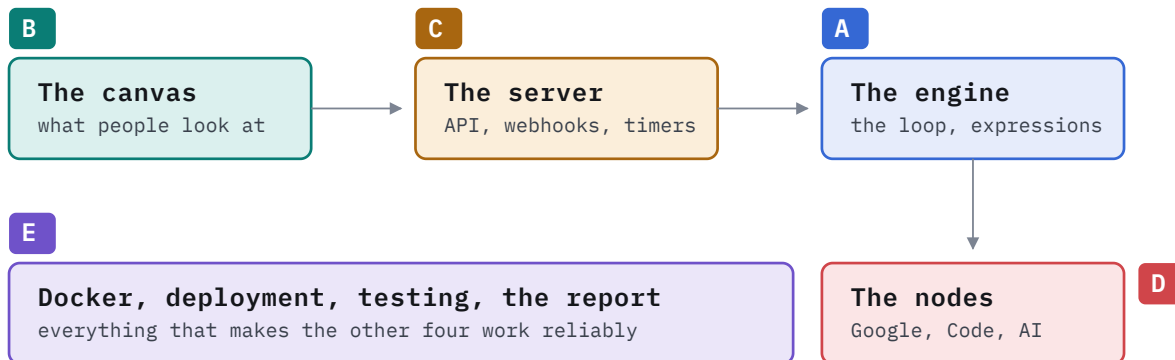
We have run this entire checklist twice, on the deployed address, on two different machines — and the fallback video exists.

THE LAST THING TO SAY

Every row above maps to a sentence in the brief. If we can tick all eleven, the demo is finished — everything else is polish, and polish is not what is being marked.

4.1 Who owns what

We own **parts of the system**, not tasks. That way nobody has to ask whose decision something is, and we rarely edit the same file at the same time.



Two written agreements hold it together: the node shape, frozen in Phase 1, and the five addresses, frozen in Phase 2.

Figure 21. With six people, D splits in two: Sheets and Drive on one side, Gmail, Docs and the AI node on the other.

STREAM A

The engine

Owns the loop, the item model and expressions. Writes Phase 1 alone, then supports everyone else. This is the technical-lead seat, and the one place two people editing at once produces a mess.

STREAM B

The canvas

Owns everything in the browser: the drawing surface, the settings-panel generator, the run viewer, the code box. The largest single piece of work in the project.

STREAM C

The server

Owns the five addresses, webhook handling, the schedule timers and the Google permission flow. The trickiest debugging in the project lands here.

STREAM D

The nodes

Owns every integration. Cannot start properly until Phase 3 is finished — before that, writes node descriptions and does the Google Cloud setup.

STREAM E

Infrastructure, testing and the report

Owns Docker, deployment, the test setup and the benchmark, and is editor-in-chief of the report and director of the demo. Starts on day one, not in Week 12 — the report is written a section a week from Week 5 onwards.

THE SEQUENCING MISTAKE TO AVOID

Do not put five people on nodes in Week 5. Until the skeleton exists there is nothing to build against, and parallel work on an unstable core produces five rewrites. Weeks 4–5 are A and C building the spine while B builds the canvas, D writes node descriptions and does the Google setup, and E stands up Docker and the automatic checks. Real parallel work starts once Phase 3 lands.

4.2 Three risks that end projects, and six habits that save them

Google keys expire after seven days

While our app is in Testing mode the long-lived key stops working after a week. Connected in Week 9, dead by Week 14 — and it looks like a broken node, not an expired key.

What we do: reconnect every Google account on the morning of the demo. It is step one of the checklist, not a footnote.

Gmail permissions are restricted

Making the app public needs a Google security review we cannot complete this semester.

What we do: stay in Testing mode and add every member — plus our demonstrator — as test users in Week 7, not in Week 8 when we first need it.

Nothing works on demo day

Campus wifi, an API outage, a laptop that decides to update. All of it happens, and always at nine in the morning.

What we do: record a full video of the working demo in Week 13, bring a second laptop, and have a phone hotspot ready.

Six habits that will save us weeks

- 01 **Build the boring version first.** An HTTP node that only does the simplest request is worth more in Week 6 than a perfect one in Week 9, because everything downstream can start.
- 02 **One person owns the engine.** Two people editing it at once produces a mess nobody can untangle.
- 03 **We are not building user accounts.** Logins and password resets earn no marks here and will eat a week.
- 04 **When a node will not work, test the plain request in Postman first.** Most “my node is broken” turns out to be a missing permission or a wrong header — nothing to do with our code.
- 05 **Print the data between nodes, constantly.** Almost every confusing bug is the shape of the data, not the logic — and section 1.3 explains the shape it should have.
- 06 **Copy n8n's good ideas deliberately, and say so in the report.** Explaining *why* a design is right is exactly what the assignment asks for. Copying without understanding is what costs marks.

IF WE FALL BEHIND

We cut *nodes*, never phases. Losing the Drive node costs almost nothing — it is one row in the acceptance checklist. Losing Phase 3 means every later node needs a hand-built settings panel and only one person can work at a time, which would cost us the whole of Weeks 8 to 10. The order of what to drop, if it comes to it: Drive, then Docs, then the second half of the benchmark, then the Code node. Never the AI node, and never a phase.

4.3 What each of us does this week

Everything on this page can be finished inside one week, and Phase 1 cannot start properly until it is.

WHO	TASK	BLOCKS
A	Write the engine script from Figure 5 and walk the whole group through it. Everything else waits on this.	everything
B	Get a React Flow canvas running that can add a box and connect two boxes. Nothing needs to save yet.	Phase 2
C	Postgres in a container, the two tables, and a server that answers one address.	Phase 2
D	Create the Google Cloud project , switch on the four APIs, fill in the consent screen, add everyone as test users.	Phase 5
E	Repository set up, the shared Docker file, automatic checks on every change, and an AWS account with a spending alarm.	Phases 2 and 6
All	Read Parts 0 and 1 of this document before Monday, and put the Week 13 rehearsal in the calendar now.	—

WHY D'S TASK IS ON THIS LIST

It looks like it belongs in Week 8. It is here because the Google permission screen has a waiting period we do not control, restricted permissions we cannot argue with, and a test-user list that must include our demonstrator. Fifteen minutes now removes the single most common cause of a failed demo in this kind of project.

OUR WEEKLY RHYTHM FROM HERE

Monday, thirty minutes: agree this week's "done when".
Thursday, thirty minutes: everyone shows what actually runs, and one person clones the repository fresh and starts it. Nothing else. Two meetings a week is enough if both end in something running.

THE ONE SENTENCE TO REMEMBER

Every phase ends in something we can run and show. If a phase takes twice as long as this document says, that is completely normal — we cut a node, never a phase, and we never start a new phase before the previous one's "done when" is actually true.